CP – 3407 Advanced Software Engineering

Group 14

Team Members:

Yuan Xin Chong

Zhao Youyang

Thura Aung

Contents

Project Overview 4
Project Objectives 4
Version Control 5
System Scope 6
Main Features 6
Customer Module 6
Restaurant Module 7
Technology Stack 7
System Architecture 7
Expected Outcomes 8
Project Development 8
1. Planning and Requirement Analysis 9
2. User Stories 9
3.Team Capacity and Feasibility Calculation 10
4.Feature Planning, Prioritisation and Delivery Strategy 11
5.Design 11
6. Implementation / Code 15
7. Frontend Development 24
8. Backend Development 25
9. Payment Integration 25
10. Order Tracking and Delivery Workflow 26
11. Testing 26
11.1 Testing Strategy 26
11.2 User Stories as Test Specifications (Test-Driven Development) 27
11.3 Automated Testing 29
11.4 Strategy Pattern 29
11.5 Mock Object Framework 30
11.6 Summary 30
12. Bug Tracking and Error Reporting 31
12.1 Examining and Improving Bug Tracking Process 31
12.2 Use of User-Story GitHub Pages for Bug Tracking 31
12.3 Use of GitHub Issues and Other Tools 32
12.4 Identified Bugs and Fixes 32
12.5 GitHub Repository Structure 33
12.6 Summary 33

13. Building and Development Tools 34
14. Agile Software Engineering 34
15. Project Technical Writing 34

Project Overview

This project is an Online Food Ordering and Delivery System that aims to simulate how real platforms like Grab Food or Food panda work. The goal is not just to build a simple ordering website, but to recreate the full flow of a food delivery service, from browsing restaurants to receiving an order. Because of we use stripe to simulate online payment, so stripe needs the secret key and public key in our config.php file. But GitHub prevents this from being pushed.

```
// STRIPE CONFIG
$stripe_secret_key = "sk_test_51TEPpvPX3z94BU2JYnebLE9THggxzhCewiBanGTE7WSHrlo96DQj64UXZVR2b5EsazhNp90JdA0E1Mec1OyyaJFJ00ghur2B5J";

$stripe_publishable_key = "pk_test_51TEPpvPX3z94BU2JJm60DbrmGb3FGnehfwCtgil5TawfJcYPoD25P66APajphrYjCx7CXUnkdPKEqY2CxRWjvzKj007rKjlStI";
```

Users can start by exploring different food categories and viewing restaurants. Users can select items and add them to a cart and choose to add or remove items. This can help users to manage their own items easily. The system supports a typical ordering process where users confirm their order by entering delivery details such as address and phone number and then proceed to payment. Stripe is used to simulate online payment, making the process more realistic without storing sensitive card information.

After placing an order, users can track its progress through different stages, such as pending, preparing, and on the way. The system also includes a delivery tracking feature, where rider information and location updates are shown to give a better sense of a real delivery experience.

Another important part of the system is order history. Instead of just storing basic order information, the system keeps detailed records of each order, including the items, prices, and quantities at the time of purchase. This means that even if the menu changes later, the order history will still show the original details correctly. The system also records status updates over time, allowing users to view how their order progressed from start to finish.

Overall, this project focuses on building a more complete and realistic food delivery system by combining user-friendly features with structured data design and practical workflow simulation.

Project Objectives

The main objectives of this project are:

Build a simple and user-friendly website for food ordering

Allow users to easily browse restaurants with menus and search for restaurants through key words

Items added by users can be automatically added to the shopping cart for easier management of selected items.

After selecting their items, users can place an order, and a rider will be randomly assigned to them.

Support secure online payments using Stripe

Implement a full workflow:

Restaurant Browsing -> Cart Management -> Order Placement -> Payment Processing -> Order Confirmation -> Food Preparation -> Rider Assignment -> Delivery Tracking -> Order Completion

Help restaurants efficiently receive and manage orders

Enable customers to track order status in real time

Design a system similar to real-world food delivery platforms

Build a system that can be extended with future features

Version Control

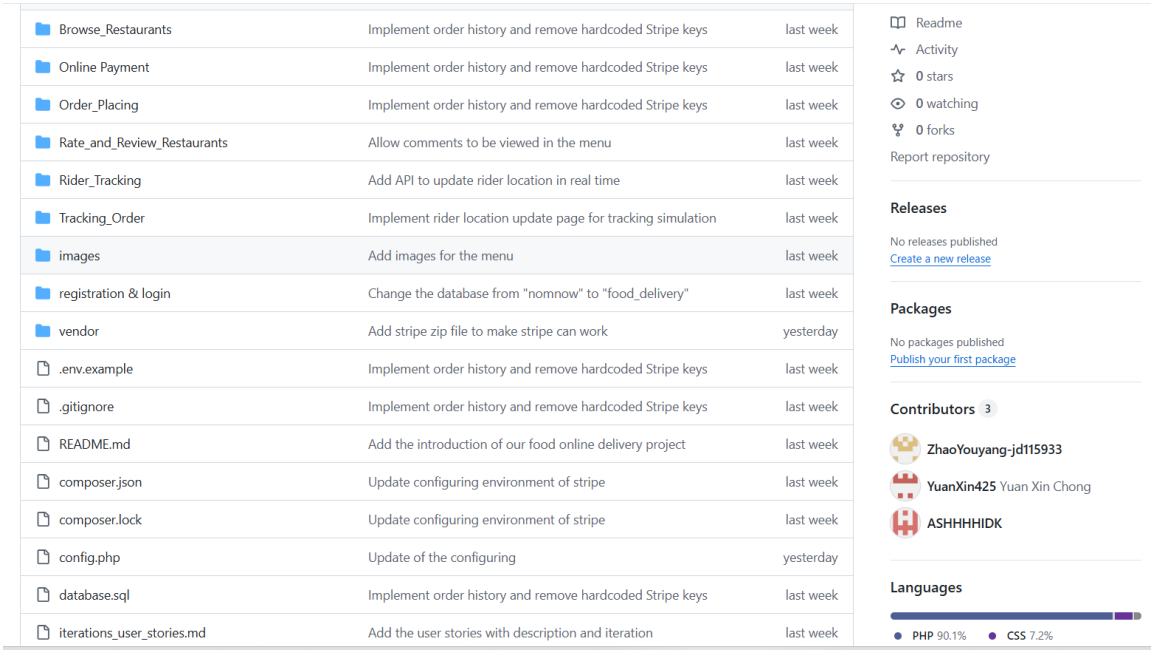
https://github.com/Yuxuanzhang9090/CP3407.git

GitHub was used as the main version control system to manage code, track changes, and support team collaboration throughout the project. All team members worked on a shared repository and committed updates regularly, ensuring that progress across different iterations could be clearly tracked and managed. Meaningful commit messages were used to describe specific changes, such as implementing new features or fixing bugs. This made it easier to understand the development history and quickly identify the purpose of each update.

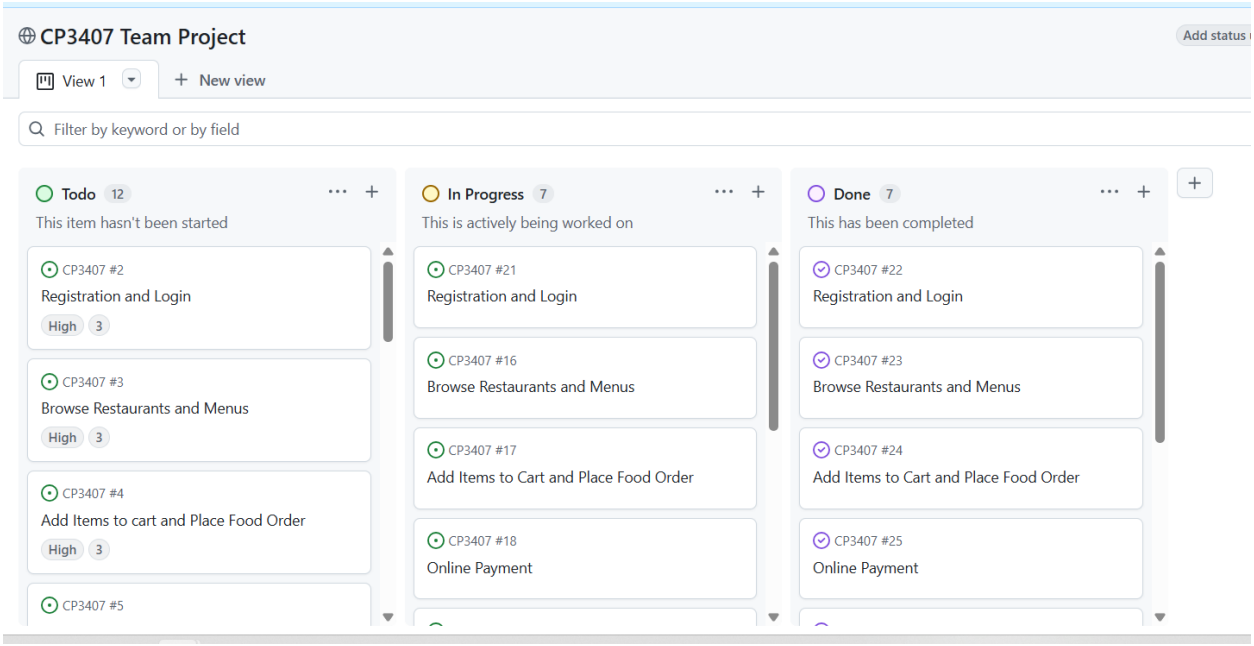
GitHub was also used to support bug tracking through Issues, where identified bugs were recorded, monitored, and marked as resolved. This provided clear traceability between problems and their corresponding fixes.

In addition, user stories and tasks were managed using a simple workflow structure (To Do, In Progress, Done), allowing the team to track development progress and ensure that features were implemented in a structured and organised manner.

Overall, GitHub played an important role in supporting version control, task management, and debugging, ensuring a smooth and controlled development process.



Browse_Restaurants	Implement order history and remove hardcoded Stripe keys	last week
Online Payment	Implement order history and remove hardcoded Stripe keys	last week
Order_Placing	Implement order history and remove hardcoded Stripe keys	last week
Rate_and_Review_Restaurants	Allow comments to be viewed in the menu	last week
Rider_Tracking	Add API to update rider location in real time	last week
Tracking_Order	Implement rider location update page for tracking simulation	last week
images	Add images for the menu	last week
registration & login	Change the database from "nomnow" to "food_delivery"	last week
vendor	Add stripe zip file to make stripe can work	yesterday
.env.example	Implement order history and remove hardcoded Stripe keys	last week
.gitignore	Implement order history and remove hardcoded Stripe keys	last week
README.md	Add the introduction of our food online delivery project	last week
composer.json	Update configuring environment of stripe	last week
composer.lock	Update configuring environment of stripe	last week
config.php	Update of the configuring	yesterday
database.sql	Implement order history and remove hardcoded Stripe keys	last week
iterations_user_stories.md	Add the user stories with description and iteration	last week



System Scope

Included Scope

- Restaurant and menu browsing
- Shopping cart and checkout system
- Order submission and storage
- Online payment integration (Stripe)
- Order status tracking
- Order history
- Basic rider assignment logic
- Database design and system integration

Main Features

Customer Module

- Browse restaurants and menus
- View food details (price, description)
- Add items to cart and manage quantity and items
- Enter delivery details (address, phone, notes)
- Place orders through checkout
- Make secure payments via Stripe
- View order history to achieve System Scope
- Track order status in real time

Restaurant Module

- Receive customer orders
- View order details (items, quantity, delivery info)
- Update order status (e.g., preparing, ready for pickup)
- Order Management System
- Create and store orders in the database

When a user places an order, the system generates a new order record and saves key information such as user details, restaurant, delivery address, and payment status.

Record detailed order items

Each order stores a list of items, including item names, quantities, and prices at the time of purchase. This ensures that order history remains accurate even if menu data changes later.

Calculate order costs automatically

The system calculates the subtotal based on selected items and adds delivery fees and service fees to generate the final total amount.

Manage the full order status lifecycle

Orders go through multiple stages such as pending, preparing, on the way, and delivered. These status changes are updated in real time and recorded for tracking purposes.

Track order progress and delivery

The system supports rider assignment and location tracking, allowing users to monitor delivery progress.

Maintain complete order history

All orders are stored and can be retrieved by users at any time, including full details such as items, prices, status timeline, and delivery information.

Technology Stack

Frontend: HTML, CSS, JavaScript
Backend: PHP
Database: MySQL
Server: XAMPP
Payment: Stripe API
Version Control: GitHub

System Architecture

This system is designed based on a three-tier architecture, which separates the system into the presentation layer, application layer, and data layer. This structure helps organize the system clearly and makes it easier to maintain and extend.

The presentation layer (frontend) is responsible for handling user interaction and displaying information through web pages. It includes features such as browsing restaurants, viewing menus, managing the cart, placing orders, and tracking deliveries. This layer focuses on providing a smooth and user-friendly experience.

The application layer (backend) manages the core logic of the system. It handles processes such as order creation, payment processing through Stripe, updating order status, and managing delivery workflows. It also connects the frontend with the database, ensuring that user actions are properly processed and reflected in the system.

The data layer (database) stores all system data, including users, restaurants, menu items, orders, and transactions. It also maintains detailed records such as order items, status history, and delivery tracking data, which support features like order history and real-time tracking.

Overall, this layered architecture allows the system to separate responsibilities clearly, making it more scalable, easier to debug, and better aligned with real-world web application design.

Expected Outcomes

By the end of the project, the system delivers a complete and functional food ordering workflow that closely reflects a real-world delivery platform. Users are able to browse restaurants and menus, add items to a cart, and place orders by providing delivery details such as address, phone number, and notes. The system processes payments through Stripe and records the payment status, ensuring a realistic transaction flow.

Once an order is placed, a full order record is created in the database, including both overall order information and detailed item-level data. Each order stores a snapshot of item names, quantities, and prices at the time of purchase, ensuring that order history remains accurate even if menu data changes later. The system automatically calculates the total cost, including subtotal, delivery fees, and service fees, providing a clear breakdown of charges.

The system also manages the complete lifecycle of an order, updating its status through stages such as pending, preparing, on the way, and delivered. These status changes are recorded over time, allowing the system to represent the full progression of each order. In addition, delivery functionality is supported through rider assignment and location tracking, enabling users to monitor delivery progress in a more interactive way.

A key outcome of the system is the implementation of a more realistic order history feature. Instead of simply listing past orders, the system preserves complete order records, including item details, pricing, delivery information, and status timelines. This allows users not only to review their past orders accurately but also supports features such as reordering and detailed order inspection, similar to real food delivery platforms.

Project Development

This project aims to develop a realistic web-based online food ordering and delivery system. The system is designed to simulate the workflow of a real food delivery platform, where customers can browse restaurants, select menu items, place orders, make online payments, and track the delivery process.

Unlike a very basic food ordering website, our project is planned as a more complete platform that connects three main parties: customers, restaurants, and delivery riders. Therefore, the project will not

only focus on ordering food, but also on payment handling, order management, rider assignment, and order tracking.
To make the project manageable and well-organized, the development process is divided into several phases.

1. Planning and Requirement Analysis

In the first phase, the team will identify the main problem and define the core functions of the system. The main goal is to make food ordering more convenient for customers and more efficient for restaurants. At this stage, the team will:

- decide the project scope and topic
- identify the main users of the system
- study how real food delivery websites work
- list the core features that must be included in the prototype

The team has already decided that the system should support:

- restaurant and menu browsing
- shopping cart
- order placement
- secure online payment
- order tracking
- order history
- restaurant-side order handling
- rider-side delivery updates

This phase is important because it provides a clear direction for the development of the entire system.

2. User Stories

Title	Short Description	Priority	Estimated Days
1. Registration and Login	I should register and log in to the system so that I can access the app and order food.	High	8 days
2. Browse Restaurants and Menus	I want to browse nearby restaurants and view their menus so that I can decide what food to order.	High	8 days
3. Add Items to cart and Place Food Order	I want to select food items, add them to a cart, and place an order so that I can purchase meals from nearby restaurants.	High	14 days
4. Online Payment	I want to pay online securely so that I can complete my order conveniently.	High	12 days
5. Track Order	I want to track my order status so that I can find out when my food will arrive.	Medium	12 days
6.View Order History	I want to view my previous orders so that I can reorder them easily in the future.	Medium	6 days

7. Rate and Review Restaurants	I want to rate and review restaurants so that I can share my experiences with other users.	Low	6 days
8. Categorize the restaurants and food	I want to choose restaurants through the different category	Low	4 days

Justification

Registration and Login: This feature is essential as it enables user authentication and secure access to the system. It interacts with the database to manage user accounts and is required before users can place orders. Therefore, it is prioritized as high since it forms the foundation of all user-related functionality.

Browse Restaurants and Menus: This feature allows users to explore available restaurants and menu items, which is necessary before making any order decisions. It requires retrieving and displaying dynamic data from the database. It is prioritised as high because it directly supports the core ordering process.

Add Items to cart and Place Food Order: This is the core transaction feature of the system. It integrates cart management, database storage, and order creation, forming the main workflow of the application. Without this feature, the system cannot function. Therefore, it is prioritized as high.

Online Payment: This feature enables users to complete transactions through secure online payment using Stripe. It integrates external payment services and manages payment status within the system. It is prioritized as high because it is required to simulate a complete and realistic food delivery platform.

Track Order: This feature allows users to monitor order progress through different status stages. It relies on order status updates and delivery tracking logic, improving transparency and user experience. It is prioritized as medium because it enhances the system but is not required for basic ordering functionality.

Rate and Review Restaurants: This is an optional feature that allows users to share feedback on their experience. It involves simple input handling and database storage. It is prioritized as low since it does not affect the core functionality of the system.

View Order History: This feature enables users to view past orders with detailed information, including item names, quantities, and prices at the time of purchase. It ensures data consistency and reflects real-world system behavior. It is prioritized as medium because it improves usability and system realism.

Categorize the restaurants and food: This feature improves usability by allowing users to filter restaurants and menu items by category. It requires additional filtering logic and minor database adjustments. It is prioritised as low since it enhances convenience but is not essential for core functionality.

3.Team Capacity and Feasibility Calculation

Total workload of all user stories

8 + 8 + 14 + 12 + 12 + 4 + 6 + 6 = 70

The project is planned for a team of 3 members over a 3-month period, which is approximately 60 working days. Therefore, the theoretical maximum capacity is:

3 × 60 = 180 person-days

However, not all available time can be spent on direct development, as time is also required for communication, debugging, testing, integration, and documentation. Following a more realistic productivity assumption of 60%, the team's adjusted development capacity is:

3 × 60 × 0.6 = 108 person-days

The revised total estimated workload for the selected user stories is 70 person-days, which is well within the team's realistic capacity of 108 person-days. This indicates that the selected set of features is achievable within the given time and resource constraints. It also leaves sufficient time for testing, refinement, and issue resolution, which strengthens the feasibility of delivering the system on time and within budget.

4.Feature Planning, Prioritisation and Delivery Strategy

The set of features was carefully planned and prioritised to ensure that the system can be delivered effectively within the available time and resource constraints. Each user story was initially estimated based on its technical complexity, including database operations, business logic, and system integration requirements. The total estimated development time exceeded the available timeframe, making it necessary to prioritise features in a structured and justified manner.

Following a customer-focused planning approach, features were organised based on their importance to the core functionality of the system. High-priority features, including user registration and login, browsing restaurants, placing orders, and payment processing, were identified as essential baseline functionality. These features are critical for the system to operate and therefore were scheduled for implementation first.

Medium-priority features, such as order tracking, reviews, and low-priority like additional usability improvements, were considered non-essential for initial delivery and were planned for later development stages. This prioritisation ensures that development effort is focused on delivering a fully functional system before extending it with additional enhancements.

The implementation order was determined based on logical dependencies between features. For example, user authentication must be completed before order placement, and order functionality must be established before integrating payment processing. This structured sequencing reduces development risk and ensures efficient integration across system components.

By aligning feature selection, prioritisation, and implementation order with the available timeframe, the project demonstrates a realistic and achievable development plan. This ensures that the system delivers essential functionality on time and within scope, fulfilling the objective of providing a practical IT solution that meets user needs while remaining manageable within given constraints.

5.Design

After identifying the requirements, the next step is to design the overall system structure. This includes database design, page structure, and workflow design.

The system is designed around a realistic food delivery process:

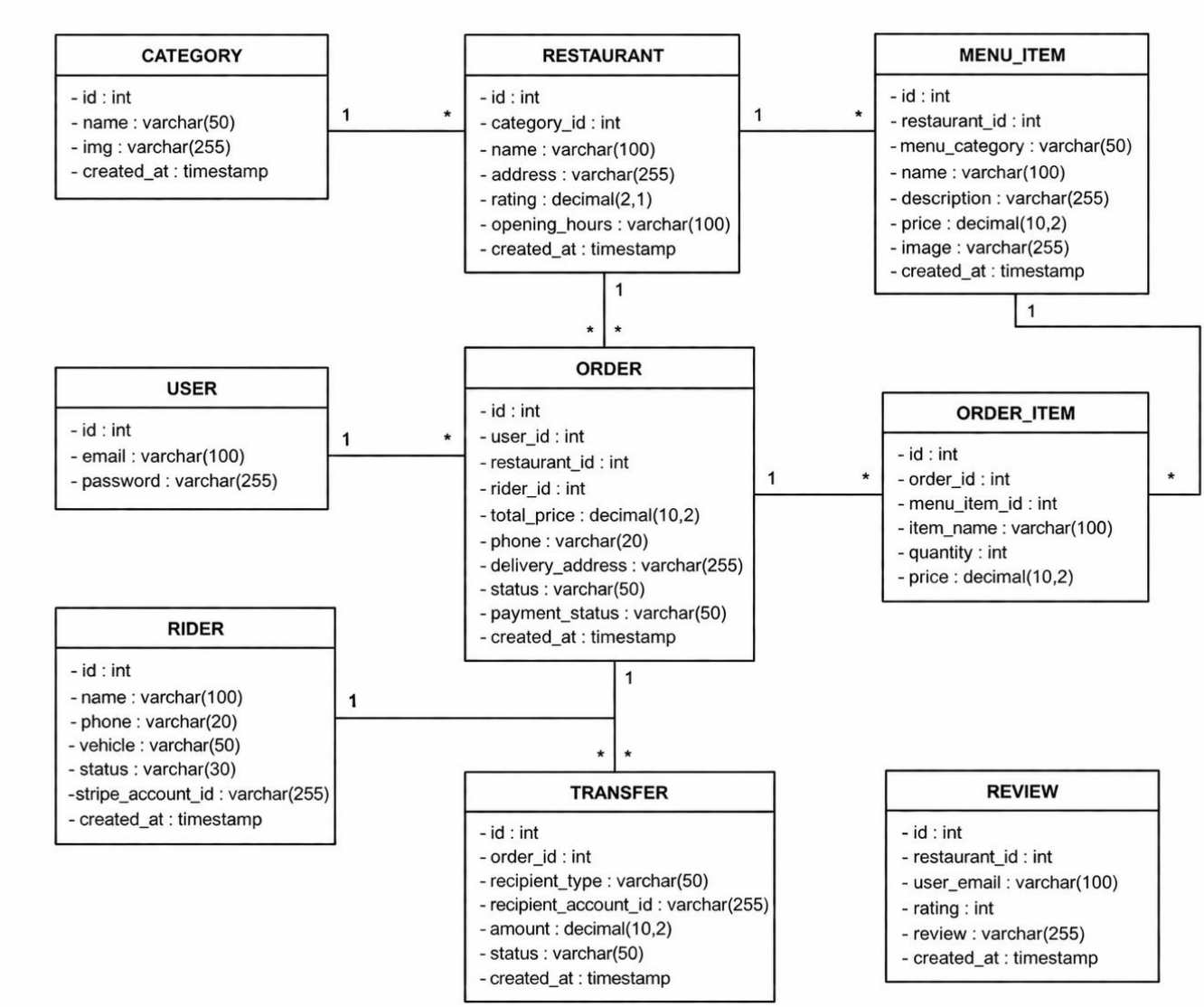
- The user browses restaurants and menus
- The user adds items into the cart
- The user confirms the order and enters delivery details
- The system calculates the total amount, delivery fee, and service fee
- The user completes payment through Stripe

- The restaurant receives the order and prepares the food
- A rider is assigned to deliver the order
- The customer tracks the delivery status until the order is completed

At this stage, the team will design:

- the page navigation flow
- the ordering workflow
- the payment workflow
- the tracking workflow
- the database tables for users, restaurants, menu items, orders, order items, riders, and payments

This Architectural diagram shows the relationship between users, orders, restaurants, and payments.



Database Design (UML Explanation and Justification)

The database design of this system is structured to support the complete workflow of an online food delivery platform, including user management, restaurant browsing, order processing, payment handling, and delivery tracking. The design follows a relational model where each entity represents a specific

component of the system, ensuring clear data organization and efficient interaction between different modules.

The USER and ORDER tables form a one-to-many relationship, where each user can place multiple orders. This allows the system to track user activities and retrieve personalized information such as order history. This design reflects real-world usage, where customers interact with the system through repeated transactions over time.

The restaurant-related structure is divided into three tables: CATEGORY, RESTAURANT, and MENU_ITEM. Each category can contain multiple restaurants, and each restaurant can have multiple menu items. This hierarchical structure supports flexible browsing and filtering, allowing users to explore restaurants by category and dynamically view menus, which improves both usability and scalability of the system.

The ORDER and ORDER_ITEM tables are separated to represent transactions and their detailed contents. One order can contain multiple items, and each item stores its own quantity and price. This design avoids data redundancy, supports accurate total calculations, and follows normalization principles, making the system more efficient and maintainable.

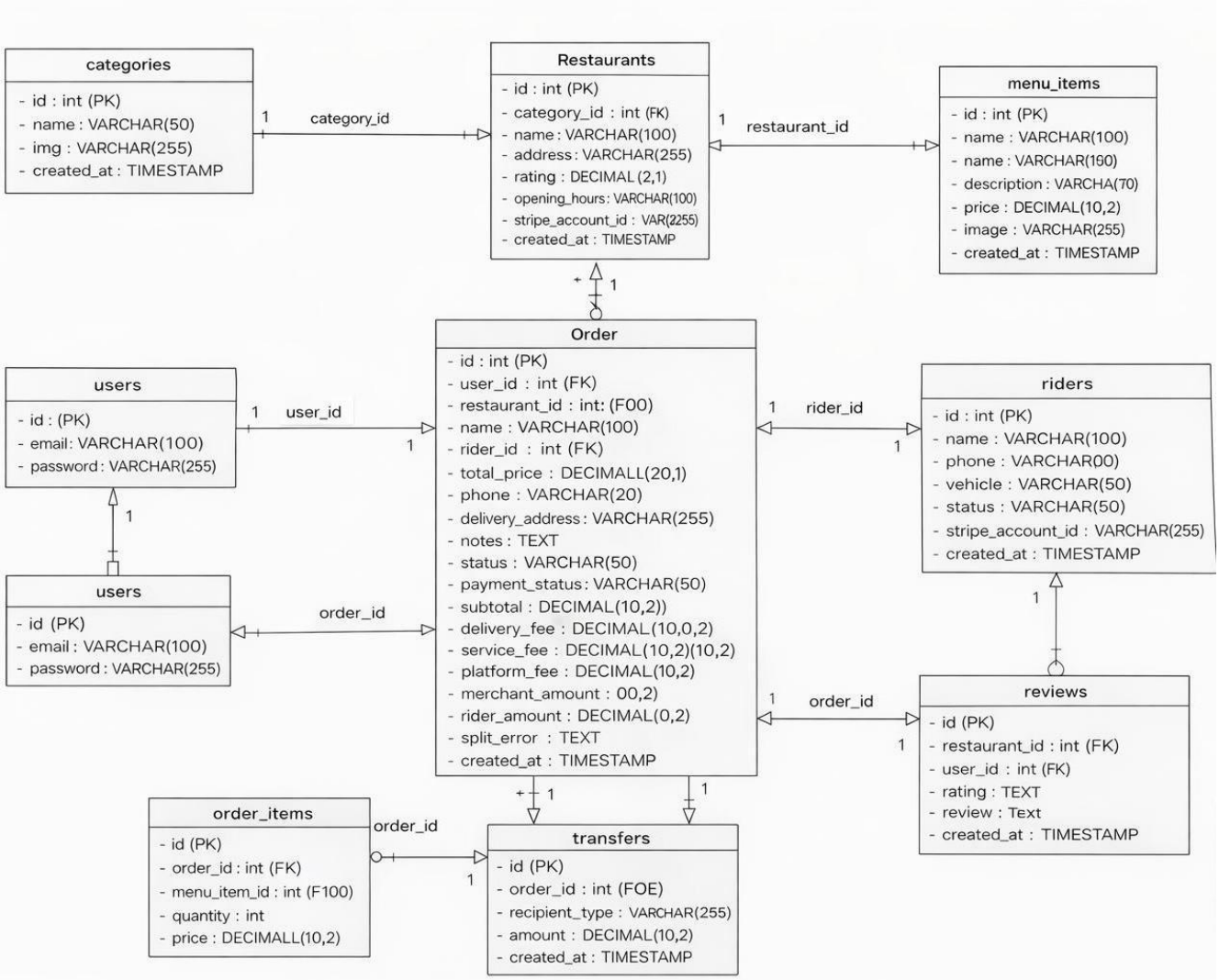
The RIDER table is linked to the ORDER table, where each order is assigned to a rider. This enables delivery management and tracking functionality within the system. By separating rider data into its own table, the system can manage delivery assignments independently and support future extensions such as rider availability or performance tracking.

The TRANSFER table is used to manage payment distribution across different recipients, such as restaurants, riders, and the platform. This design supports more realistic financial workflows by allowing the system to record how payments are split, which is essential for integrating external payment services like Stripe and simulating real-world delivery platforms.

The REVIEW table stores user feedback for restaurants, linking users to their submitted ratings and comments. This allows the system to support a review and rating mechanism, improving user experience by enabling customers to make informed decisions based on previous feedback.

Overall, the database design follows key principles such as normalization, scalability, and data integrity. By separating data into logically structured tables and maintaining clear relationships, the system ensures consistent data management while supporting complex operations such as ordering, payment processing, and delivery tracking. This design also closely reflects real-world processes, making the system both practical and extensible.

ERD Design



The overall database design follows a structured relational approach to model real-world operations of a delivery system.

Firstly, the system separates core entities such as users, restaurants, orders, and menu items into independent tables. This reduces data redundancy and ensures that each entity can be managed and updated separately without affecting others.

Secondly, the use of linking tables such as order_items allows the system to support one-to-many relationships, where a single order can contain multiple menu items. This reflects real-world ordering behaviour and improves flexibility.

In addition, the orders table acts as the central hub that connects users, riders, and restaurants. This design simplifies data retrieval and ensures that all key information related to an order is logically grouped.

The inclusion of financial fields (such as platform_fee, merchant_amount, and rider_amount) and the transfers table support tracking of payment distribution. This makes the system closer to real-world platforms where revenue needs to be split across different stakeholders.

Overall, this design ensures that the system is scalable, maintains data integrity, and accurately represents real-world relationships between different components of the system.

6. Implementation / Code

Divide Iterations and User Stories into Tasks

Iteration 1 focused on delivering the baseline functionality of the system. Users were able to register, browse restaurants, view menus, and place orders. This ensured that a minimum working system was achieved early.

1.Registration and Login

Tasks： Design user table (USER)
Create registration form
Validate input (email, password)
Hash password and store in database
Implement login authentication (PHP)
Create session management
Handle login errors (wrong password, user not found)

2. Browse Restaurants and Menus

Tasks: Create CATEGORY and RESTAURANT tables
List all the restaurants and add menus for the restaurant
Connect the restaurant with its own menu page
Add search bar for searching restaurants

3. Add to Cart & Place Order

Tasks: Create and manage cart session
Add, update, and remove cart items
Display cart contents and calculate total price
Recalculate item subtotal after quantity change
Build checkout form for delivery details
Save order and order items into database, then clear cart

Iteration 2 extended the system by adding payment integration and order tracking. This allowed users to complete transactions and monitor their order status, making the system more realistic and complete.

4. Online Payment

Task: Create Stripe checkout session based on order details (amount, order ID)
Connect the place order with stripe
Direct user to Stripe-hosted payment page
Handle payment success and return to the system (success page)
Verify payment and update payment status in database
Link payment information to the corresponding order

5. Track Order

Task: Retrieve order details and current status from the database (orders table)
Display order status progression
Fetch and update order status dynamically using backend APIs
Store and display status history using order_status_history table
Retrieve rider information and location from order_tracking table

Visualize rider movement on a map and update location over time
Iteration 3 focused on enhancing user experience by adding features such as reviews, order history, and category-based browsing. These features improve usability but are not essential for core system functionality.

6. View Order History

Task: Retrieve past orders based on the logged-in user (user_id)
Display a list of orders with key information (order ID, restaurant, total price, status, date)
Allow users to view detailed information for each order (items, quantities, prices)
Link each order with its corresponding order items (order ↔ order_item)
Display order status for each order
Connect order history with payment part and view details
Provide reorder functionality for previous orders
Handle empty order history (no orders case)

7. Rate and Review

Task: Create review form for users to submit rating and comments
Validate review input before submission (review table)

Link each review to the corresponding restaurants
Display submitted reviews on the menu page

8.Categorize the restaurants and food

Task: Organise restaurants into categories (e.g. fast food, drinks, desserts)
Assign each restaurant to a specific category
Retrieve and display restaurants based on selected category
Display category options for users to browse
Filter restaurant list according to selected category
Categories menu items within each restaurant

Iterative Development Process (Iteration 1 Focus)

Task Progress Monitoring (Iteration 1)

User Story	Task	Status
Registration & Login	Design user table (USER)	Done
Registration & Login	Create registration form	Done
Registration & Login	Validate input (email, password)	Done
Registration & Login	Hash password and store in database	Done
Registration & Login	Implement login authentication (PHP)	Done
Registration & Login	Create session management	Done
Registration & Login	Handle login errors (wrong password, user not found)	Done
Browse Restaurants	Create CATEGORY and RESTAURANT tables	Done

Browse Restaurants	List all the restaurants and add menus for the restaurant	Done
Browse Restaurants	Connect the restaurant with its own menu page	Done
Browse Restaurants	Add search bar for searching restaurants	In Progress
Add to Cart & Place Order	Create and manage cart session	Done
Add to Cart & Place Order	Add, update, and remove cart items	Done
Add to Cart & Place Order	Display cart contents and calculate total price	Done
Add to Cart & Place Order	Recalculate item subtotal after quantity change	In Progress
Add to Cart & Place Order	Build checkout form for delivery details	Done
Add to Cart & Place Order	Save order and order items into database, then clear cart	Done

Calculate Actual Velocity (Iteration 1)
The estimated workload for Iteration 1:
Login (8 days)
Browse (8 days)
Place Order (14 days)
E1 = 30 user-story days
Due to additional time required for setup and debugging:
E2 = 28 user-story days
Estimated velocity:
v1 = 0.7
Actual velocity:
v2 = (E2 / E1) × v1
v2 = (28 / 30) × 0.7 = 0.65
The lower velocity indicates that the team underestimated the effort required for initial system setup and integration.

Velocity Visualization (Burndown Chart Interpretation)
The burndown chart shows the reduction of remaining work across the iteration.
Start: 70
Iteration 1: 48 (above ideal -> slower progress)
Iteration 2: 13 (below ideal -> faster progress)
Iteration 3: 0
In Iteration 1, the actual progress is slightly slower than the ideal line, which corresponds to the task tracking table where some tasks remained In Progress.
This indicates that many tasks were still being worked on rather than completed during the early phase.
As development progressed, more tasks transitioned to Done, resulting in faster progress in later stages.

Updating Iteration 2 Backlog Using Velocity
Based on the calculated velocity (v2 = 0.65), the team adjusted the workload for Iteration 2.
Original Plan
Online Payment (12 days)
Order Tracking (12 days)
Total = 24 days

Adjustment Based on Velocity
Since the team's actual capacity was lower than expected:
Priority was given to core functionality (payment + tracking logic)
Non-essential features (UI refinements, extra feedback messages) were delayed
Tasks were broken down more carefully to improve tracking and completion
This ensured that the workload was aligned with the team's actual development capacity rather than initial assumptions.
Final Integration
Overall, the iteration demonstrates a complete agile development cycle, where tasks were tracked, progress was measured using velocity, and future planning was adjusted accordingly. This reflects a realistic and adaptive software development process.

Iteration 2 – Velocity and Backlog Adjustment
Task Progress Monitoring (Iteration 2)
During Iteration 2, the team focused on online payment and order tracking, which were critical features identified from user feedback in Iteration 1.

User Story	Task	Status
Online Payment	Create Stripe checkout session based on order details (amount, order ID)	Done
Online Payment	Connect the place order with stripe	Done
Online Payment	Direct user to Stripe-hosted payment page	Done
Online Payment	Handle payment success and return to the system (success page)	In Progress
Online Payment	Verify payment and update payment status in database	Done
Online Payment	Link payment information to the corresponding order	Done
Track Order	Retrieve order details and current status from the database (orders table)	Done
Track Order	Display order status progression	Done
Track Order	Fetch and update order status dynamically using backend APIs	In Progress

Track Order	Store and display status history using order_status_history table	Done
Track Order	Retrieve rider information and location from order_tracking table	In Progress

Several tasks remained in progress due to integration complexity, especially for real-time tracking and payment confirmation.

Velocity Calculation (Iteration 2)

Planned workload:

Online Payment (12 days)

Order Tracking (12 days)

$E1_2$ = 24 user-story days

Actual completed work:

$E2_2$ = 20 user-story days

Using previous velocity:

v2 = 0.65

Updated velocity:

$v3 = (E2_2 / E1_2) \times v2$

$v3 = (20 / 24) \times 0.65 \approx 0.54$

The reduced velocity reflects the increased complexity of integrating external systems (Stripe) and implementing real-time tracking features.

Burndown Interpretation (Iteration 2)

The burndown chart shows a sharp drop during this iteration:

48 -> 13

This indicates that:

many core features were completed

tasks transitioned quickly from In Progress -> Done

However, some tasks remained incomplete due to:

API integration challenges

real-time tracking complexity

Backlog Adjustment for Iteration 3

Based on the reduced velocity ($v3 \approx 0.54$), the team adjusted Iteration 3 planning.

Planned Features

Order History (6 days)

Review System (6 days)

Category Browsing (4 days)

Total = 16 days

Adjustment Strategy

prioritized simple, low-risk features first

reduced complexity of implementation

focused on usability improvements instead of heavy backend logic

This ensured that the remaining work could be completed within the team's reduced capacity.

Iteration 3 – Final Completion Phase
Task Progress Monitoring

User Story	Task	Status
View Order History	Retrieve past orders based on the logged-in user (user_id)	Done
View Order History	Display a list of orders with key information (order ID, restaurant, total price, status, date)	Done
View Order History	Allow users to view detailed information for each order (items, quantities, prices)	Done
View Order History	Link each order with its corresponding order items	Done
View Order History	Display order status for each order	Done
View Order History	Connect order history with payment part and view details	Done
View Order History	Provide reorder functionality for previous orders	Done
View Order History	Handle empty order history (no orders case)	Done
Rate and Review	Create review form for users to submit rating and comments	Done
Rate and Review	Validate review input before submission	Done
Rate and Review	Store review data in the database (review table)	Done
Rate and Review	Link each review to the corresponding restaurant	Done
Rate and Review	Display submitted reviews on the menu page	Done
Categorize the restaurants and food	Organise restaurants into categories (e.g. fast food, drinks, desserts)	Done
Categorize the restaurants and food	Assign each restaurant to a specific category	Done
Categorize the restaurants and food	Retrieve and display restaurants based on selected category	Done

Categorize the restaurants and food	Display category options for users to browse	Done
Categorize the restaurants and food	Filter restaurant list according to selected category	Done
Categorize the restaurants and food	Categories menu items within each restaurant	Done

All tasks were successfully completed in this iteration (all the tasks of all the user stories).
Final Velocity Observation
Since all tasks were completed:
velocity stabilized
The team achieved full completion by focusing on simpler features and reducing technical complexity.

Final Burndown Alignment
13 -> 0
This confirms:
all remaining work completed system fully delivered

FINAL Agile Cycle Summary
Across all three iterations, the team followed a complete agile development cycle:
Iteration 1: Built core system (slower progress due to setup)
Iteration 2: Implemented complex features (variable velocity due to integration challenges)
Iteration 3: Completed usability features (stable and efficient completion)
Task tracking, velocity calculation, and backlog adjustment were continuously used to guide development decisions.
This demonstrates a realistic and adaptive approach to software development, where planning is refined based on actual performance rather than initial assumptions.

User Feedback and Improvement of Each Iteration
Iteration 1 - Core System Delivery
In the first iteration, the system delivered the baseline functionality, including user registration and login, browsing restaurants, viewing menus, and placing orders. A working version of the system was deployed and demonstrated.

User Feedback: Users were able to complete the basic ordering process, but reported that the system felt incomplete. There was no payment functionality, and after placing an order, users could not see what was happening next.

Improvements: Based on this feedback, the next iteration focused on completing the transaction flow by adding payment functionality and introducing order status tracking, so users could both complete their orders and understand their progress.

Iteration 2 -Payment and Tracking Integration
In the second iteration, online payment using Stripe was implemented, and order tracking functionality was introduced. This allowed users to complete transactions and monitor the progress of their orders.

User Feedback: Users found the payment process functional, but feedback indicated that the system needed clearer feedback of successful status or fail status after payment and payment details. For tracking, users wanted more detailed and understandable updates instead of basic status changes.

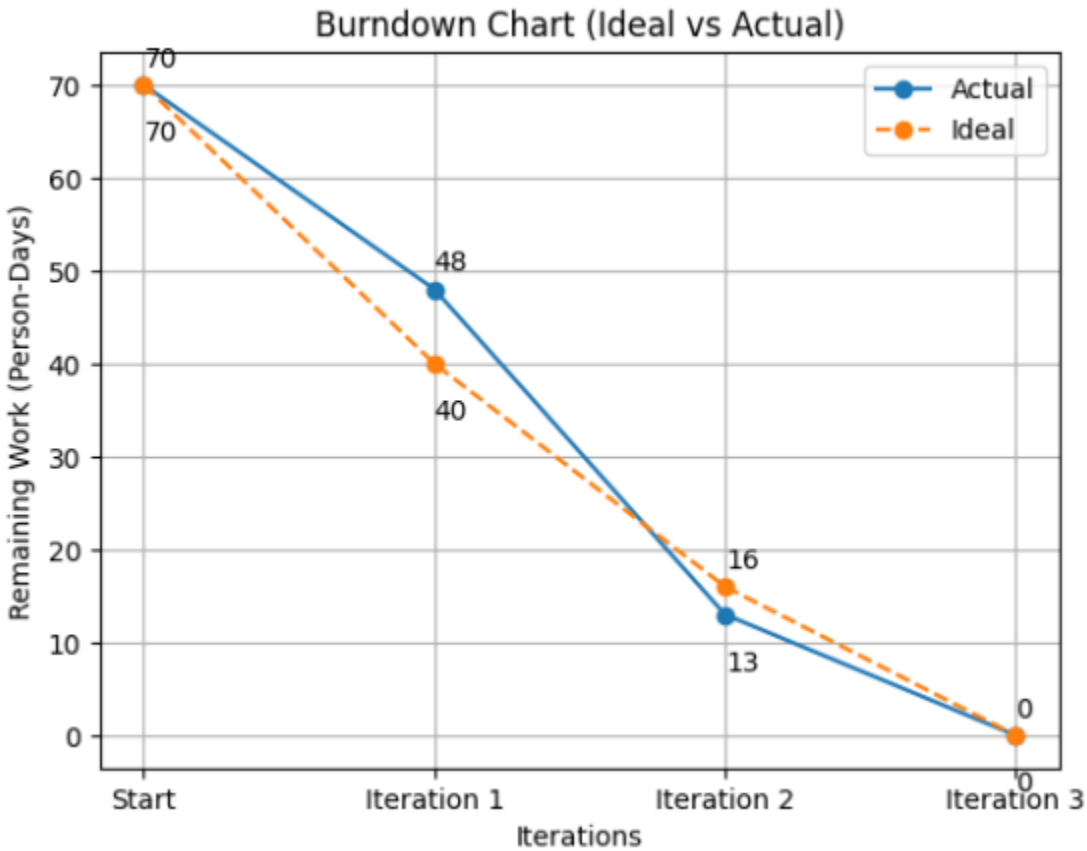
Improvements: The system was improved by refining the order process into more detailed status stages, allowing users to clearly understand each step. The tracking interface was enhanced and add information of different order status to visually reflect these status updates. In addition, a map-based feature was introduced to display the rider's location, providing a more intuitive and realistic tracking experience.

Iteration 3 -User Experience Enhancement
In the final iteration, features such as order history, restaurant reviews, and category-based browsing were implemented to improve usability and user interaction.

User Feedback: Users found the system more complete, but suggested improvements to navigation and convenience, especially when trying to revisit previous orders or explore restaurants more efficiently.

Improvements: The system was enhanced by adding an order history page that allows users to view past orders and their details. Review functionality was introduced to enable users to provide feedback details rather than only selecting the grade of stars in restaurants. Category-based browsing was also added to help users quickly find relevant restaurants. Additional UI adjustments were made to improve layout clarity and navigation flow.

Burndown Chart and Iteration Alignment



The burndown chart reflects the progress of the project across three iterations and aligns closely with the development stages and user feedback collected during each phase. The chart demonstrates how the workload was reduced in a realistic and adaptive manner.

Iteration 1 – Core System Delivery

In Iteration 1, the team focused on delivering the core system functionality, including registration and login, browsing restaurants, viewing menus, and placing orders .

Although a working system was successfully delivered, the progress was slightly slower than the ideal burndown line. This is reflected in the chart where the remaining workload is higher than expected.

This deviation is reasonable because the team spent additional time on:

setting up the database structure

implementing authentication and session management

ensuring the core ordering flow works correctly

This explains why progress in Iteration 1 was slower than ideal.

Iteration 2 – Payment and Tracking Integration

In Iteration 2, the team implemented online payment (Stripe) and order tracking features .

This iteration shows a sharper drop in the burndown chart, where the actual progress falls below the ideal line. This indicates that the team completed more work than initially planned.

The faster progress can be explained by:

reuse of the existing order structure
clearer system architecture established in Iteration 1
focused implementation of two major features which are tracking order and online payment. These two main features highlighted the need for clearer payment confirmation and more detailed tracking updates .
This iteration represents a productivity peak where major functional gaps were successfully closed.

Iteration 3 – User Experience Enhancement

In Iteration 3, the remaining features were implemented to enhance usability, including order history, reviews, and category-based browsing .
The burndown chart shows that the remaining work was completed steadily, bringing the workload to zero.
These features were less technically complex but focused on improving user experience:
enabling users to revisit previous orders
allowing users to leave reviews
improving navigation through categories
This iteration represents the refinement phase of the project.

Overall Burndown Interpretation

The variation between the ideal and actual burndown lines demonstrates a realistic development process. Progress was slower in the initial iteration due to system setup and foundational work, accelerated in the second iteration when key features were implemented, and stabilised in the final iteration during system refinement.
This confirms that the team followed a logical and adaptive development strategy, prioritising core functionality first, then integration, and finally user experience enhancements.

7. Frontend Development

The frontend part of the system focuses on user interaction and page design. The aim is to make the website easy to use and similar to a real food delivery platform.
The main pages planned for development include:
Home page / categories page
Restaurant listing page
Restaurant menu page
Cart page
Checkout / place order page
Payment page
Order success page
Track order page
My orders page
Login / register pages
The frontend should clearly guide users through the ordering process. For example, users should be able to easily move from menu browsing to cart review, from cart to checkout, and from payment to order tracking.
In addition, the interface should provide clear status information, such as:
order placed

preparing
out for delivery
delivered
This helps improve the overall user experience.

8. Backend Development

The backend is one of the most important parts of this project because it handles the system logic and connects the frontend with the database.
The backend development will include:

- user authentication and session handling
- cart management
- order creation and storage
- total price calculation
- order item insertion
- restaurant and rider assignment
- payment processing integration
- order status updates
- order history retrieval
- tracking information display

For this project, PHP is used to handle the server-side logic, while MySQL is used to store and manage all system data.
The backend should ensure that:

- orders are stored correctly
- food items in each order are recorded properly
- payment status is updated correctly
- users can retrieve their previous orders
- tracking information reflects the latest order progress

This phase is crucial because even if the frontend looks good, the website will not function properly without a strong backend structure.

9. Payment Integration

A major feature of this project is secure online payment. Instead of only simulating a payment process, the team plans to integrate Stripe so that the website is closer to a real commercial platform.
The payment module will support:

- checkout session creation
- redirection to Stripe payment page
- payment confirmation
- payment success display
- storage of payment-related information in the database

This part is especially valuable because it shows that the project goes beyond a simple academic demo. It demonstrates how real food delivery platforms process transactions securely.

The team also plans to structure the system in a way that supports future improvements, such as:

- storing Stripe session IDs
- recording payment status
- preparing for restaurant and rider payout logic
- allowing the platform to calculate service fees and commission

This makes the system more realistic and professionally designed.

10. Order Tracking and Delivery Workflow

To make the website more practical, order tracking is planned as an important system feature.
After an order is placed and paid, the customer should be able to check the progress of the order. The tracking function is expected to include several stages such as:

- order received
- restaurant preparing food
- rider assigned
- on the way
- delivered

This feature is important because food delivery is not only about placing orders. Customers also want to know what is happening after payment.
The system will therefore include:

- a track order page
- a my orders page
- status update logic in the backend
- the possibility of rider-side status updates

This part makes the system feel much closer to a real food delivery website and improves the completeness of the project.

11. Testing

11.1 Testing Strategy

Testing was conducted throughout all iterations to ensure that each component of the system functions correctly both independently and as part of the overall workflow. Since the system involves multiple connected stages (browse -> cart -> order -> payment -> tracking -> view order history), any failure in one component can affect the entire process.
To address this, testing was structured into four levels:
Unit Testing
Used to verify individual logic such as cart calculations, checkout validation, payment status updates, order status transitions, and category filtering.
Integration Testing
Ensures different modules work together correctly, such as:

cart -> checkout -> order creation
order -> payment -> database update

payment -> tracking -> order status update

System Testing (End-to-End)

Simulates the full user journey:

browsing restaurants
placing an order
completing payment
tracking the order

Acceptance Testing

Validates that all implemented features meet the requirements defined in the user stories.

This structured approach ensures that the delivered implementation matches the planned functionality.

11.2 User Stories as Test Specifications (Test-Driven Development)

In this project, user stories were used as the basis for test specifications following a Test-Driven Development (TDD) approach. Instead of writing code first, expected behaviours were derived directly from user stories and translated into test cases before implementation. This means development was guided by requirements rather than assumptions. By doing this, the team could make sure that each feature was built to satisfy a clear user need, and that testing was directly linked to the planned functionality of the system. The selected user stories below were used as test specifications because together they represent the full user journey of the system, from browsing restaurants and placing an order, to payment, tracking, and reviewing past orders.

User Story - Browse Restaurants and Menus (Iteration 1)

As a customer, I want to browse restaurants and view menus so that I can choose what to order.

This user story leads to the following test specifications:

all available restaurants should be displayed correctly
selecting a restaurant should open its corresponding menu page
menu items should be displayed with the correct name and price
the search function should return relevant restaurants
invalid or empty search input should not break the system

This user story is important because browsing is the starting point of the ordering process. If restaurant lists, menu pages, or search results are incorrect, the user cannot continue to the later steps of the system.

User Story - Add to Cart and Place Order (Iteration 1)

As a customer, I want to add items to a cart and place an order so that I can purchase food.

This user story leads to the following test specifications:

items should be added to the cart correctly
updating item quantity should recalculate subtotal and total price
removing items should update the cart correctly
checkout should only proceed when valid delivery details are provided
a valid order should be saved in the database
the cart should be cleared after successful order placement

This user story represents the core transaction flow of the system. It is one of the most critical parts to test because errors in cart handling or order saving would directly prevent customers from making purchases.

User Story - Online Payment (Iteration 2)

As a customer, I want to pay online securely so that I can complete my order conveniently.

This user story leads to the following test specifications:
a valid order should be able to initiate a payment process
the system should redirect the user to the Stripe-hosted payment page
successful payment should update the order status to “paid”
cancelled or failed payment should not mark the order as paid
payment information should be correctly linked to the corresponding order
This user story is important because payment is a key part of the system’s transaction process. If the payment result is not handled correctly, the system may show incorrect order states or fail to continue to the next stage.

User Story - Track Order (Iteration 2)

As a customer, I want to track my order so that I can know the delivery progress.
This user story leads to the following test specifications:
the current order status should be displayed correctly
status updates should follow the correct order progression
status history should be stored and displayed correctly
rider information and location should match the current order
invalid or unauthorised access should be rejected
This user story is important because it gives users visibility after payment is completed. Even if ordering and payment work properly, the system would still feel incomplete if customers cannot see what is happening to their order.

User Story - View Order History (Iteration 3)

As a customer, I want to view my past orders so that I can review or reorder easily.
This user story leads to the following test specifications:
past orders should be retrieved based on the logged-in user
the order list should display correct information such as order ID, restaurant, total price, status, and date
users should be able to open and view detailed order information
each order should correctly link to its order items
the system should handle the case where no order history exists
This user story extends the usability of the system and improves the overall customer experience. It is also useful for testing whether order records are stored and retrieved correctly after the main transaction has been completed.

TDD Process Applied

The development process followed three main TDD steps:
Write the test first
The expected behaviour was identified from the user story and written as a test case before implementation.
Implement the minimum code
Only enough code was written to make the test pass.
Refactor the code
After the test passed, the code was improved and cleaned up while keeping all tests working.
For example, in the Online Payment feature, the expected behaviour that “successful payment should update the order status to paid” could first be written as a test. The payment update logic will then be

implemented until the test is passed. After that, the code could be refactored to improve structure, such as separating payment gateway handling from order to update logic.
Similarly, for Track Order, tests could first define that order statuses must follow the correct sequence, and that invalid order access should be rejected. The implementation would then be written to satisfy those requirements.
By applying this process, the team ensured that each implemented feature was directly based on user requirements, and that testing was built into development rather than added only at the end.

11.3 Automated Testing

To improve reliability and reduce manual effort, automated testing was implemented using the PHP Unit. Core logic was separated into service-level functions (order calculation, payment handling, tracking validation), allowing them to be tested independently without relying on full page execution.
Automated tests include:
cart subtotal and total price calculation
checkout validation (empty cart, invalid restaurant)
payment success and failure handling
order status transition validation
tracking history record creation
restaurant category filtering
A total of 12 automated tests were implemented, exceeding the minimum requirement of 10 tests.
Appropriate test datasets were used, including:
valid and invalid cart data
successful and failed payment scenarios
valid and invalid status transitions
category filtering with matching and non-matching data
These tests ensure correctness of business logic and reduce the risk of system errors.

11.4 Strategy Pattern

The Strategy pattern is a behavioural design pattern that allows different algorithms or behaviours to be encapsulated into separate classes and selected dynamically at runtime.
In this project, the Strategy pattern can be applied to the payment module. For example, if the system supports multiple payment methods in the future (such as Stripe, cash on delivery, PayPal), each method can be implemented as a separate strategy:
Stripe Payment Strategy
Cash On Delivery Strategy
PayPal Strategy
The system can then select the appropriate strategy without relying on complex conditional statements. This improves code flexibility, maintainability and scalability

11.5 Mock Object Framework

A Mock Object Framework is used in testing to simulate external dependencies, allowing the system to be tested in isolation.

In this project, several components depend on external systems, such as Stripe payment gateway, database operations, session data and tracking and location updates

Directly testing these dependencies is not practical, as it would be slow, unreliable, and difficult to control. Therefore, mock objects can be used to simulate these dependencies. For example, a mock payment service can simulate successful or failed payment responses without calling the real Stripe API.

This allows faster and more reliable tests, controlled testing scenarios and verification of system behaviour without external interference

Mock objects are especially useful in supporting TDD, as they enable testing business logic independently from external systems.

11.6 Summary

Overall, the testing approach in this project was designed to ensure both correctness and completeness across all system components. By structuring testing into unit, integration, system, and acceptance levels, the team was able to verify not only individual functions but also the interactions between different modules throughout the full user workflow.

A key strength of this approach is the use of user stories as test specifications within a Test-Driven Development (TDD) process. Instead of treating testing as a separate phase after implementation, expected behaviours were derived directly from user requirements and translated into test cases before development. This ensured that all implemented features were aligned with user needs and reduced the risk of missing or incorrect functionality.

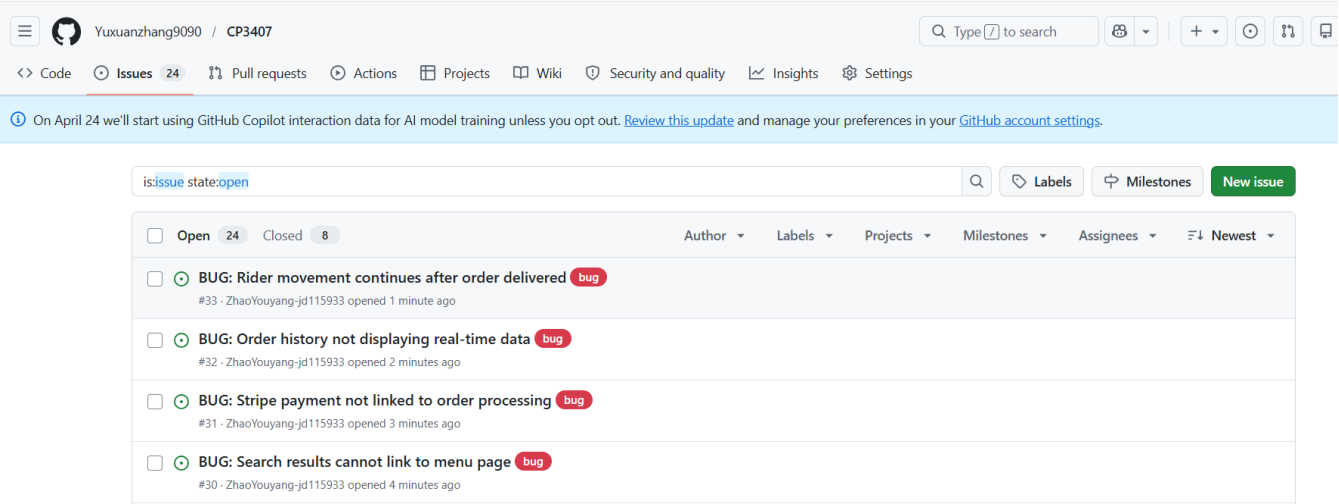
In addition, automated testing using PHP Unit was applied to validate critical business logic, including order processing, payment handling, tracking updates, and category filtering. The use of appropriate test datasets, including both valid and edge-case scenarios, further strengthened the reliability of the system.

From a design perspective, the exploration of the Strategy pattern demonstrates how the system can be extended to support multiple payment methods in a scalable and maintainable way. Similarly, the use of mock objects enables testing to be performed independently from external services such as Stripe, improving both efficiency and test reliability.

Most importantly, the selected user stories collectively cover the complete user journey, from browsing and ordering to payment, tracking, and reviewing past orders. This ensures that testing is not limited to isolated features but instead validates the system as an integrated solution.

Therefore, the combination of structured testing levels, TDD practices, automated testing, and supporting design techniques results in a robust and well-tested system that closely matches the planned implementation and delivers reliable functionality to users.

12. Bug Tracking and Error Reporting



12.1 Examining and Improving Bug Tracking Process

During system testing in Iteration 3, several bugs were identified across different parts of the system, particularly in search navigation, payment integration, order history, and order tracking. Initially, these issues were recorded informally within the team, which made it difficult to track their status and ensure all bugs were properly resolved.

To improve this process, the team adjusted their bug reporting approach by introducing a structured format. Each bug was documented with a clear description, steps to reproduce, expected and actual results, severity level, and resolution status. This made it easier to analyse problems, identify root causes, and prioritise fixes based on their impact on the system.

This adjustment transformed bug tracking from simple notes into a systematic process, making debugging more efficient and aligned with real-world software development practices.

12.2 Use of User-Story GitHub Pages for Bug Tracking

To meet the minimum requirement, bugs were recorded and linked to the relevant user stories. Each issue identified during testing was associated with the feature it affected, ensuring a clear relationship between requirements and defects.

For example:

The issue where search results could not link to the correct menu page was associated with the Browse Restaurants and Menus user story

The payment integration issue was linked to the Online Payment user story

The problem with static order history was related to the View Order History user story

The tracking inconsistency was linked to the Track Order user story

By organising bugs under their corresponding user stories, the team ensured traceability between system requirements, testing outcomes, and debugging actions. This made it easier to understand how each bug impacted specific parts of the system.

12.3 Use of GitHub Issues and Other Tools

In addition to linking bugs to user stories, GitHub Issues was used as the main tool for tracking and managing bugs. Each identified bug was created as a separate issue with a structured format, including description, reproduction steps, expected and actual results, and status.
Using GitHub Issues provided several advantages:
improved organisation of bugs compared to text-based notes
clear visibility of bug status (open, in progress, resolved)
ability to track fixes through commits and updates
better collaboration among team members
The team also considered using GitHub Projects to manage bugs in a Kanban-style workflow (To Do, In Progress, Done), which would further improve task tracking and team coordination in larger-scale development.

12.4 Identified Bugs and Fixes

During system testing, several important bugs were identified and resolved:

Search Results Cannot Link to Menu Page

During testing of the search functionality, it was found that restaurants returned from the search bar could not correctly link to their corresponding menu pages. When users searched for a restaurant and selected a result, the system either failed to navigate or opened an incorrect page. This issue disrupted the user flow at an early stage, preventing users from continuing to browse menus and place orders.
The cause of this issue was incorrect handling of restaurant identifiers in the search result links. The system did not properly pass the restaurant ID to the menu page, resulting in broken navigation. To resolve this, the link generation logic was updated to ensure that each search result included the correct restaurant ID. After the fix, users were able to successfully navigate from search results to the correct menu pages.

Order Processing and Stripe Payment Not Connected

A critical issue was identified in the integration between order processing and Stripe payment. Although users were able to place orders, the system frequently failed when transitioning to the payment stage. In some cases, the Stripe checkout session was not created correctly, and in others, the order status was not updated after payment.
This issue broke the transaction flow, as orders remained in a “pending” state even after payment attempts. The root cause was inconsistent data handling between the order processing module and the Stripe integration, particularly involving incorrect or missing order IDs during payment processing.
To fix this, the process_order logic and Stripe integration were revised to ensure that order details were correctly passed, validated, and linked throughout the payment process. After this fix, successful payments correctly updated the order status to “paid,” allowing the system to proceed to the tracking stage.

Order History Not Updated Dynamically

During testing of the order history feature, it was found that the page displayed static data instead of real-time order information. Even after placing new orders, the order history did not reflect recent transactions, making the feature inaccurate and misleading.
This problem occurred because the initial implementation used hardcoded data and was not connected to the database. As a result, the system failed to retrieve actual user orders.

To resolve this issue, the order history page was updated to dynamically retrieve data from the database using the logged-in user's ID. Orders were correctly linked with their corresponding items and payment status. After the fix, the order history page displayed up-to-date and accurate information, improving both system reliability and user experience.

Tracking Status and Rider Movement Not Synchronized

Another issue was observed in the order tracking feature, where the rider's movement did not match the order status. Specifically, even after the order was marked as "delivered," the rider's position continued to update on the map.

This created inconsistency between the system state and the visual representation, which could confuse users and reduce trust in the tracking feature. The cause of this issue was that the rider location update logic was not properly linked to the order status conditions.

To fix this, a condition was introduced to stop updating the rider's position once the order status reached "delivered." After implementing this change, the tracking behaviour became consistent with the actual order lifecycle.

12.5 GitHub Repository Structure

The project repository was organised into clearly structured folders to support development and testing. These include:

core system modules (ordering, payment, tracking)

database scripts and schema

documentation and user stories

This structure allows the lecturer to easily navigate the repository and understand how different components of the system are organised.

12.6 Summary

The bug tracking and error reporting process in Iteration 3 ensured that all identified issues were systematically recorded, analysed, and resolved. By linking bugs to user stories and managing them using GitHub Issues, the team established clear traceability between requirements, testing, and debugging.

The bugs identified mainly involved integration issues across different components of the system, including search navigation, payment processing, database consistency, and real-time tracking. Resolving these issues significantly improved the overall reliability and usability of the system.

Overall, the adoption of structured bug tracking methods and appropriate tools ensured that the final system was stable, consistent, and aligned with the intended functionality.

11. Deployment and Demonstration Preparation

After development and testing, the final phase is system deployment and preparation for demonstration. The system will be run in a local development environment using XAMPP. The team will prepare the project so that the lecturer can clearly see:

- the website interface
- the ordering workflow
- payment integration
- order status updates
- tracking functionality
- overall system design

The team will also prepare:

- GitHub repository for code sharing and version control
- screenshots or demo scenarios
- explanation of each team member's contribution
- possible future improvements

This final step is important because a good project is not only about coding, but also about presenting the system clearly and professionally.

13. Building and Development Tools

The project utilised a range of software development tools to support implementation and testing. The main tools include PHP and MySQL for backend development, and GitHub for version control and collaboration. External libraries such as Stripe API were integrated to support secure online payment functionality.

These tools were not only used for development but also for solving real system challenges. For example, Stripe was used to handle payment processing, while debugging tools and error logs were used to identify and fix integration issues between order processing and payment modules. GitHub was also used to manage code versions, track bugs, and organise development tasks.

Overall, the combination of development tools and external libraries enabled the team to build a functional and realistic system while maintaining code quality and system reliability.

14. Agile Software Engineering

The project followed an agile development approach, where the system was built incrementally across three iterations. Each iteration focused on a specific set of features, allowing the team to deliver a working version of the system early and continuously improve it.

Iteration 1 focused on core functionality such as registration, browsing, and ordering

Iteration 2 introduced payment integration and order tracking

Iteration 3 enhanced user experience with features such as order history and reviews

User stories were used to define requirements, and tasks were organised using a simple workflow (To Do, In Progress, Done). Feedback from testing was used to identify bugs and guide improvements in subsequent iterations.

This iterative approach ensured flexibility, continuous testing, and gradual refinement of the system, reflecting key principles of agile software engineering.

15. Project Technical Writing

Technical documentation was produced throughout the project to clearly explain system design, implementation, and testing. The report includes structured sections such as user stories, system development, testing, and bug tracking, ensuring that all aspects of the project are well-documented.

GitHub was also used to support technical documentation by organising project files, code, and development history. Clear descriptions were provided for features, commits, and issues, allowing others to understand how the system was developed and improved over time.

The writing style focuses on clarity, structure, and logical explanation, ensuring that both technical and non-technical readers can understand the system and its development process.